

SCS 2312 - Take-home Assignment

23002042 - DGH THATHSARA

Parser Implementation Report

1. Context-Free Grammar (CFG)

I came up with this grammar to represent the language we need to parse. It covers variable declarations with `int`, printing with `print`, and arithmetic expressions with the usual operators.

- $\text{Program} \rightarrow \text{StmtList}$
- $\text{StmtList} \rightarrow \text{Statement StmtList} \mid \epsilon$
- $\text{Statement} \rightarrow \text{Declaration} \mid \text{PrintStmt}$
- $\text{Declaration} \rightarrow \text{'int' IDENTIFIER '=' Expression ';'}$
- $\text{PrintStmt} \rightarrow \text{'print' '(' Expression ')' ';'}$
- $\text{Expression} \rightarrow \text{Term (('+' | '-') Term)^*}$
- $\text{Term} \rightarrow \text{Factor (('*' | '/') Factor)^*}$
- $\text{Factor} \rightarrow \text{INTEGER} \mid \text{IDENTIFIER} \mid \text{'(' Expression ')'}$

The **terminals** are: `int`, `print`, `IDENTIFIER`, `INTEGER`, `=`, `+`, `-`, `*`, `/`, `;`, `(`, `)`

The **non-terminals** are: Program, StmtList, Statement, Declaration, PrintStmt, Expression, Term, Factor

The **start symbol** is Program.

The reason I split expressions into Expression, Term, and Factor is to get operator precedence right — `*` and `/` should be evaluated before `+` and `-`, and having separate rules for each level handles that automatically.

2. Token Categories

Before parsing, the source code gets broken into tokens. Here are all the types my tokeniser recognises:

Token Type	What it is	Examples
TOKEN_INT_KW	The int keyword	int
TOKEN_PRINT_KW	The print keyword	print
TOKEN_IDENTIFIER	Variable names	x, myVar, total
TOKEN_INTEGER	Number values	5, 20, 100
TOKEN_ASSIGN	The equals sign	=
TOKEN_PLUS	Plus	+
TOKEN_MINUS	Minus	-
TOKEN_STAR	Multiply	*
TOKEN_SLASH	Divide	/
TOKEN_SEMICOLON	End of statement	;
TOKEN_LPAREN	Opening bracket	(
TOKEN_RPAREN	Closing bracket	)
TOKEN_EOF	Nothing left to read	—
TOKEN_UNKNOWN	Something we don't recognise	@, #

3. How the Code Works

3.1 Tokeniser

The ``next_token`` function goes through the source code one character at a time. It skips over spaces and newlines, then figures out what kind of token it's looking at:

- Characters like ``=`, `+`, `;`, etc.` are straightforward — each one maps directly to a token.
- If it sees a digit, it keeps reading digits to build up a full number.
- If it sees a letter or underscore, it reads the whole word and then checks whether it's a keyword (``int`` or ``print``) or just a variable name.
- Anything else gets flagged as unknown.

3.2 Parser

I used a recursive descent approach, which basically means each rule in the grammar has its own function. They call each other following the grammar structure:

- ``parse_program()`` is the starting point — it grabs the first token and loops through all the statements until the file ends.
- ``parse_statement()`` looks at the current token to figure out if we're dealing with a variable declaration or a print statement.
- ``parse_declaration()`` handles lines like ``int z = x + y;``. It reads the variable name, parses the expression on the right side of ``=`, calculates the value, and stores it.`
- ``parse_print_statement()`` handles ``print(z);``. It evaluates whatever's inside the brackets and prints the result.
- ``parse_expression()`` deals with ``+`` and ``-``.
- ``parse_term()`` deals with ``*`` and ``/``.

- `parse_factor()` handles the simplest things — a number, a variable name, or a sub-expression in brackets.

The way these are nested means `**` and `/` naturally get evaluated before `+` and `-`, which is how normal math works.

3.3 Storing Variables

I used a simple array as a symbol table. Each entry stores the variable's name and its current value. When a new variable is declared with `int`, it gets added. When a variable is used in an expression, the program looks it up in the table and uses its stored value.

3.4 Error Handling

The program catches two kinds of errors:

Syntax errors — when the code doesn't follow the grammar. For example, if there's a missing semicolon, or you write something like `int y = x +;` where a number or variable is expected after the `+`.

Semantic errors — when the code is grammatically fine but doesn't make sense. Like using a variable that was never declared, or trying to declare the same variable twice.

When the program hits an error, it reports what went wrong and which line it happened on. It then tries to skip ahead to the next semicolon so it can keep going and catch any other errors in the rest of the file, rather than just stopping at the first problem.

